AWS-COST-CALCULATOR · 백엔드 부하테스트

optimize 엔드포인트가 AI 호출 동안 DB 커넥션을 인질로 잡던 문제를 해결했다

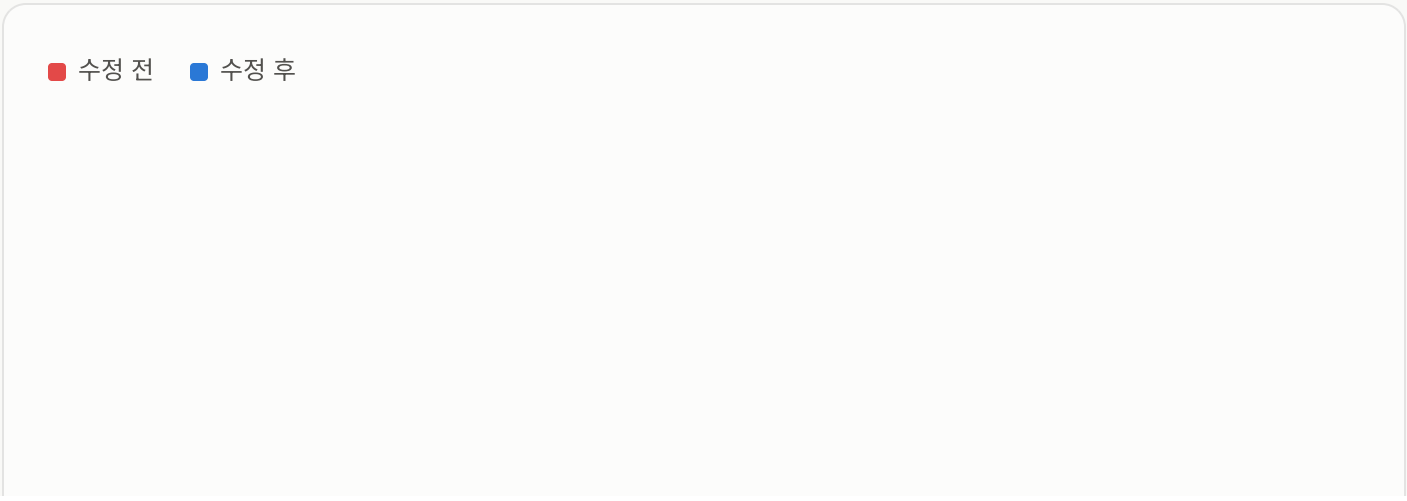
`OptimizationService.optimize()`가 외부 AI 호출을 읽기 전용 트랜잭션으로 감싸고 있어서, 외부 응답을 기다리는 내내 HikariCP 커넥션 하나를 물고 있었다. 동일한 k6 스크립트, 동일한 동시성으로 수정 전/후 코드를 각각 한 번씩 돌려 비교했다.

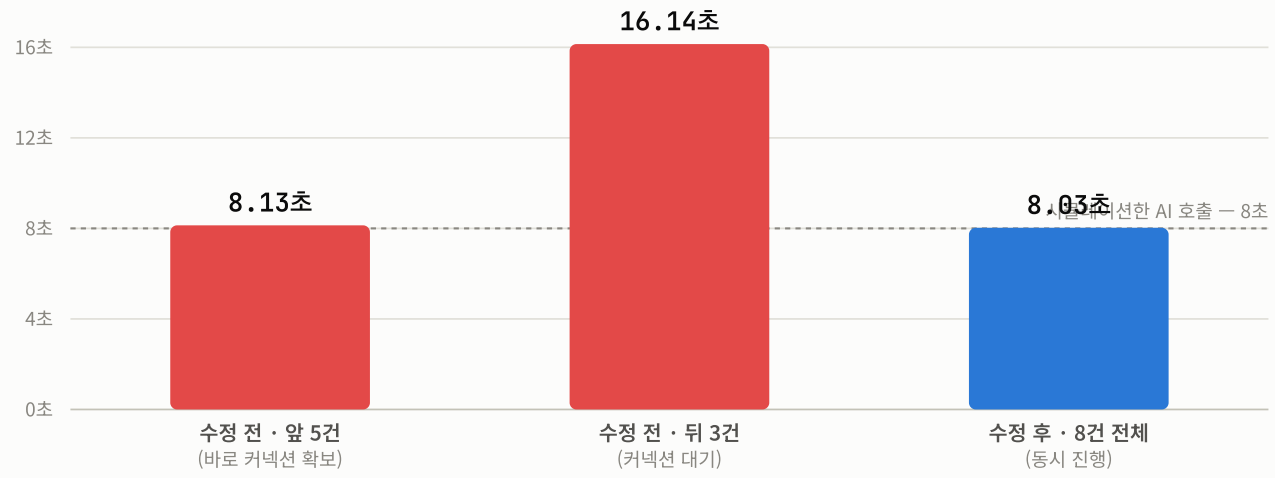
날짜 2026-09-18 **도구** k6 v1.7.1 **DB 풀** 최대 5개 커넥션 **부하** optimize 동시 8건 + 무관 데이터 읽기 3 VU

가장 느린 optimize 호출	무관한 읽기 요청 최악의 지연	요청 유실 건수 (두 실행 모두)
~~16.14초~~ → **8.03초**	~~7.53초~~ → **15ms**	**0 / 0**
두 번째 물결이 더 이상 커넥션을 기다리지 않음	`GET /api/projects`, 폭주 중 동시 진행	30초 풀 타임아웃을 안 넘겨서일 뿐 — 아래 유의사항 참고

optimize 동시 8건 요청이 실제로 겪은 일

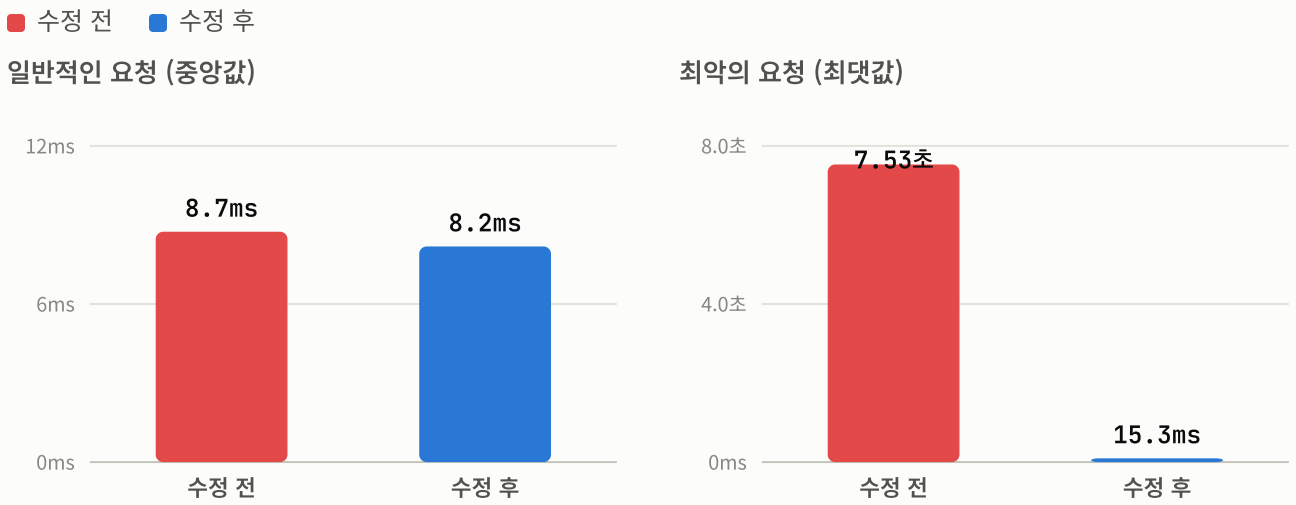
풀 최대치는 5로 설정했다. 수정 전에는 트랜잭션이 시뮬레이션한 AI 호출(8초) 내내 커넥션을 붙잡고 있었기 때문에 8건 중 5건만 바로 시작할 수 있었고, 나머지 3건은 빈 커넥션을 기다리며 8초 대기를 두 번 겪었다. 수정 후에는 AI 호출 전에 커넥션을 반납하므로 8건 모두 동시에 진행된다.





optimize와 무관한 트래픽이 입은 피해

optimize 폭주가 진행되는 동안, 서로 무관한 가상 유저 3명이 0.5초 간격으로 계속 `GET /api/projects`를 호출했다. 일반적인 지연은 거의 움직이지 않았고, 피해는 전부 꼬리 구간(tail)에 몰렸다 — 커넥션 풀이 고갈됐을 때 전형적으로 나타나는 패턴이다.



근본 원인과 수정

메서드 레벨의 `@Transactional(readOnly = true)`은 그 안의 모든 코드를 감싼다 — `adviceFor()`가 외부 AI 제공자를 호출하는 부분까지 포함해서다. 가상 스레드는 그 대기 동안 저렴하게 park되지만, 트랜잭션이 체크아웃한 HikariCP 커넥션은 그렇지 않다.

수정 전 — 커넥션을 약 8~60초 동안 붙잡음

```
@Transactional(readOnly = true)
public OptimizationResponse optimize(
    Long projectId, Long userId, String locale
) {
    Project project = projectService
        .getOwnedOrThrow(projectId, userId);
    List<Resource> resources = resourceReposit
        .findByProjectId(projectId);
    // ...비용 계산...

    CostOptimizationAdvice advice =
        adviceFor(context);  // 외부 AI 호출,
                             // 이 시점까지 tx기

    return new OptimizationResponse(/*...*/);
}
```

수정 후 — 읽기 구간에서만 커넥션 사용

```
// @Transactional 없음
public OptimizationResponse optimize(
    Long projectId, Long userId, String locale
) {
    Project project = projectService
        .getOwnedOrThrow(projectId, userId);
    List<Resource> resources = resourceReposit
        .findByProjectId(projectId);
    // 각 repo 호출이 자체적으로 트랜잭션을
    // 갖고 빠르게 반환됨

    CostOptimizationAdvice advice =
        adviceFor(context);  // 트랜잭션이 열려있지
                             // 않은 상태로 실행

    return new OptimizationResponse(/*...*/);
}
```

크레딧 차감도 대기 구간 밖으로 옮김

같은 변경으로 check-then-act 경쟁 상태도 함께 닫혔다: 기존에는 잔액을 확인하고, AI 호출 전체를 기다린 다음, 차감하는 순서였다 — 동시 요청 두 개가 차감 전에 둘 다 확인을 통과할 수 있었다. 이제는 차감이 원자적이고 선행되며, 실패 시 환불한다. 이 부분은 위 부하테스트 대상은 아니었다(폭주 실행 시 크레딧을 소모하지 않으려고 admin 우회를 사용) — 지연이 아니라 정합성을 고치는 수정이다.

TX1 · 즉시 커밋
크레딧 잔액 확인, 차감, 커넥션 반납

→

외부 AI 호출
트랜잭션 없이 가상 스레드만 대기

→

성공
결과를 그대로 반환

TX2 · 실패 시에만
크레딧 환불, 커밋

테스트 구성

외부 AI 호출	시뮬레이션, 8000ms 고정 sleep	시뮬레이션 이유	실제 Claude/GLM 호출 대비 제어 가능 & 무료
DB	로컬 Postgres 14, aws_calculator	Hikari 최대 풀	5 (이번 실행을 위해 오버라이드)

optimize_storm	8 VU × 1 iteration	background_reads	3 VU, 30초, 약 0.5초 간격
인증	admin 테스트 유저 (크레딧 체크 우회)	테스트 후 제거한 파일	LoadTestSlowOptimizationAdvisor.java

원본 수치

지표	수정 전	수정 후
optimize · 최소	8.135 s	8.026 s
optimize · 평균	11.137 s	8.027 s
optimize · p95	16.143 s	8.028 s
optimize · 최대	16.144 s	8.028 s
배경 읽기 · 평균	181.4 ms	8.2 ms
배경 읽기 · 중앙값	8.7 ms	8.2 ms
배경 읽기 · p95	11.7 ms	10.6 ms
배경 읽기 · 최대	7529.3 ms	15.3 ms
에러율 (두 실행 모두)	0%	0%

유의사항: 8초는 실행 시간을 짧게 유지하려고 고른 값이지 실제 제공자와 맞춘 값이 아니다 — 실제 Claude/GLM 호출은 설정된 120초 타임아웃에 가깝게 걸릴 수 있고, 그러면 수정 전 큐는 Hikari의 30초 커넥션 획득 타임아웃을 훌쩍 넘겨서 단순 지연이 아니라 실제 500 에러로 이어지기 시작할 것이다. 단일 머신, 단일 실행, 네트워크 변동 없음. 배경 읽기 p95가 거의 움직이지 않은 이유는 132/177건 중 1~2건 정도만 실제로 경합 구간에 걸렸기 때문이다 — 최댓값이 그 운 나쁜 요청들이 겪은 수치를 보여준다.